Stress Level Classifier: A TF-IDF Based Machine Learning Approach for Three-Class Text Stress Detection

Kerollos Emad
ID: 23100920
CSCI417 Machine Intelligence
Medical Track

Nardeen Raafat
ID: 231001098
CSCI417 Machine Intelligence
Medical Track

***Abstract*—This paper presents a supervised machine learning system for classifying short written text into three stress levels: Low, Medium, and High. The project follows the Stress Level Classifier idea in the medical track and treats the task as a multi-class natural language processing problem. A balanced dataset of 5,523 samples was constructed from a public MentalDistress dataset and additional Stack Exchange text collected through an API-based scraping workflow. The text was cleaned, deduplicated, mapped into stress-level classes, balanced, and converted into numerical features using term frequency-inverse document frequency (TF-IDF). Five machine learning algorithms were implemented and compared on the same stratified train/test split: Multinomial Naive Bayes, Logistic Regression, Linear Support Vector Machine, Stochastic Gradient Descent Classifier, and Decision Tree. Logistic Regression achieved the best overall result, with 77.19% accuracy and 77.30% macro F1-score. The results show that classical text classification models can provide a simple, interpretable baseline for stress-level prediction when the dataset is carefully prepared and evaluated.**

***Index Terms*—stress classification, text classification, TF-IDF, machine learning, natural language processing, mental health informatics**

I. Introduction

Stress is commonly expressed through short written messages, survey responses, forum posts, and journal-style entries. Automatic stress-level classification can help organize large amounts of text and support early analysis of stress-related patterns. In this project, the goal is not to diagnose a medical condition, but to build a machine learning classifier that predicts whether a text reflects Low, Medium, or High stress.

The selected project idea belongs to the medical track and focuses on classifying text responses into three stress levels. The work was completed in two phases. Phase 1 focused on collecting, inspecting, cleaning, merging, and balancing the dataset. Phase 2 focused on preprocessing the text, implementing at least five models, comparing their performance, and preparing the final paper.

The main contributions of this work are as follows:

- A final balanced dataset of 5,523 labeled text samples was prepared for three-class stress classification.
- A consistent TF-IDF preprocessing pipeline was used to transform raw text into numerical features.
- Five machine learning models were trained and evaluated using the same train/test split.
- The best model was selected based on accuracy, macro precision, macro recall, macro F1-score, and weighted F1-score.
- A simple application workflow was designed so a user can enter text and receive a predicted stress level.

II. Problem Definition

The task is a supervised multi-class text classification problem. Each input sample is a text message, and the target output is one of three classes:

- **Low:** text that shows calm, neutral, general, or low-pressure expression.
- **Medium:** text that shows frustration, worry, workload pressure, or moderate concern.
- **High:** text that shows anxiety, overwhelm, burnout, panic, or severe stress language.

Let x_i represent the ith text sample and $y_i \in \{Low, Medium, High\}$ represent its label. The model learns a function $f(x_i) = y_i$ from labeled training examples. Because the text is unstructured, it must first be converted into a numerical feature vector before training.

III. Dataset Construction

A. Data Sources

The dataset was built from two sources. The first source was the MentalDistress public dataset, which contains English text samples with mental-health-related labels. For this project, the labels *Others*, *Frustrated*, and *Anxious* were mapped to Low, Medium, and High stress respectively. The labels *Depressed* and *Suicidal* were excluded from the first version because they represent sensitive clinical content and should not be mixed directly with ordinary stress-level labels.

The second source was text collected from Stack Exchange using an API-based scraping workflow. Search queries were grouped by expected stress level, such as burnout and overwhelmed for High, worried deadline and workload stress for Medium, and calm routine or balanced schedule for Low. The scraped rows were treated as weakly labeled data and then

inspected, deduplicated, length-filtered, and merged with the main dataset.

B. Final Dataset

After cleaning and merging, the combined dataset contained 5,594 rows. To remove class imbalance, the majority classes were downsampled to the size of the smallest class. The final dataset contains 5,523 rows, with exactly 1,841 samples per class. Table I summarizes the final dataset.

TABLE I
FINAL BALANCED DATASET SUMMARY

Property	Value
Total samples	5,523
Low samples	1,841
Medium samples	1,841
High samples	1,841
Text duplicates	0
Missing text values	0
Train samples	4,418
Test samples	1,105
Primary source rows	4,419
Scraped source rows	1,104

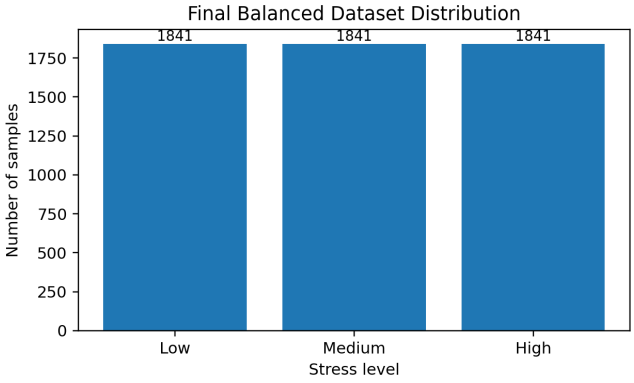

Fig. 1. Final class distribution after balancing.

IV. PREPROCESSING METHODOLOGY

The preprocessing stage was important because text data cannot be used directly by traditional machine learning models. The implemented preprocessing pipeline included the following steps.

A. Cleaning and Filtering

The dataset was checked for missing text values and duplicate samples. No missing text values and no duplicates remained in the final balanced dataset. During scraped-data cleaning, very short texts were removed because they usually do not contain enough context for stress-level classification, while extremely long texts were removed to reduce noise and training instability.

B. Label Mapping

The original labels were converted into the three final stress classes. The mapping was designed to keep the task focused on stress rather than broader mental health diagnosis. This decision also made the classification target clearer and safer for a course project.

C. Class Balancing

Balanced classes are important because accuracy alone can become misleading when one class is much larger than the others. The combined dataset had more Low samples than Medium and High samples. Therefore, each class was randomly sampled to 1,841 rows using a fixed random state. This produced equal class representation and made macro-average metrics more meaningful.

D. Text Vectorization

The raw text was converted into numerical features using TF-IDF. TF-IDF gives high weight to terms that are frequent in a document but less common across the full dataset. This is useful for stress classification because words such as *overwhelmed*, *deadline*, *worried*, or *calm* may be informative for certain classes. The vectorizer used lowercasing, English stop-word removal, unigrams and bigrams, a minimum document frequency of 2, sublinear term frequency scaling, and a maximum of 8,000 features.

V. MODEL IMPLEMENTATION

Five algorithms were trained on the same TF-IDF representation and evaluated on the same stratified 80/20 train/test split. Stratification preserved the same class proportions in training and testing.

A. Multinomial Naive Bayes

Multinomial Naive Bayes is a common baseline for text classification. It assumes that features contribute independently to the class probability. Although the independence assumption is simplified, the method is fast, interpretable, and often effective for word-frequency features.

B. Logistic Regression

Logistic Regression is a linear classifier that estimates class probabilities using weighted combinations of input features. It is suitable for sparse TF-IDF vectors because it can learn which words and phrases increase or decrease the probability of each stress class.

C. Linear Support Vector Machine

The Linear SVM searches for separating hyperplanes with large margins between classes. It is widely used for text classification because high-dimensional sparse features often become linearly separable enough for strong performance.

D. Stochastic Gradient Descent Classifier

The SGD Classifier trains a linear model using stochastic gradient updates. It is efficient for large sparse text matrices and can approximate logistic-style classification while training quickly.

E. Decision Tree

The Decision Tree model learns a hierarchy of feature-based rules. It is more interpretable than many other algorithms because predictions can be explained as a sequence of decisions. However, text data has many sparse features, so a single tree can be less stable than linear models.

VI. Evaluation Metrics

The models were evaluated using accuracy, macro precision, macro recall, macro F1-score, and weighted F1-score. Accuracy measures the total percentage of correct predictions. Macro metrics compute the metric separately for each class and then average the values equally, which is important because all stress levels should be treated as equally important. Weighted F1-score accounts for class support, but since the test set is balanced, it is close to macro F1-score.

VII. Results and Discussion

Table II shows the performance comparison across all five algorithms. Logistic Regression obtained the best result with 77.19% accuracy and 77.30% macro F1-score. The SGD Classifier was very close, with 76.84% macro F1-score. Linear SVM also performed well, while Multinomial Naive Bayes and Decision Tree were weaker but still provided useful baselines.

TABLE II
Model Performance on the Stratified Test Set

Model	Acc.	Prec.	Recall	F1
Logistic Regression	0.772	0.775	0.772	0.773
SGD Classifier	0.767	0.770	0.767	0.768
Linear SVM	0.755	0.757	0.755	0.756
Multinomial NB	0.727	0.731	0.727	0.728
Decision Tree	0.710	0.721	0.709	0.711

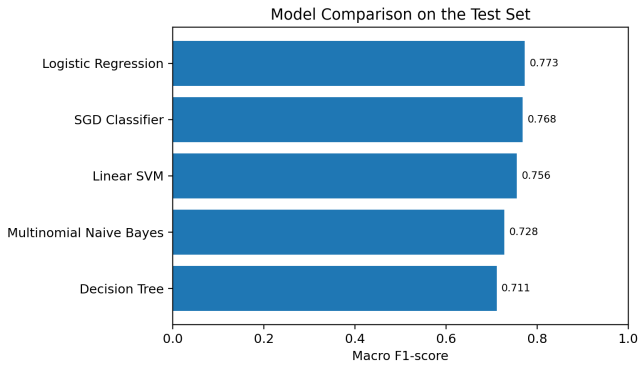

Fig. 2. Macro F1-score comparison for the five implemented models.

The confusion matrix for the best model is shown in Fig. 3. The High class achieved the strongest class-level F1-score of 0.861. The Low and Medium classes were more difficult to separate. This is expected because low stress and moderate stress can share similar wording, especially in advice-seeking

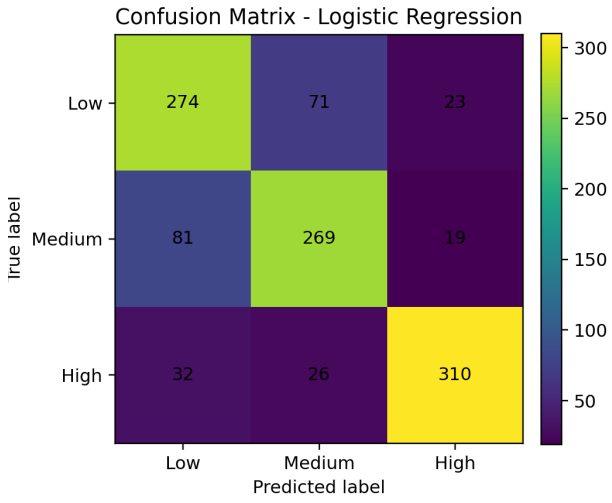

Fig. 3. Confusion matrix for the best-performing Logistic Regression model.

forum posts where a user may describe a problem calmly but still show pressure.

The results suggest that linear models are better suited to the current TF-IDF representation than a single Decision Tree. Logistic Regression and Linear SVM can use many small word-weight signals at the same time, which fits sparse text data. The Decision Tree must split on individual features, making it more sensitive to sparse word occurrences. Naive Bayes trained extremely quickly and remained useful as a simple baseline, but its independence assumption limited its final score.

VIII. Application Interface

A simple user-facing workflow was designed around the trained classifier. The user enters a short text response, the same preprocessing and TF-IDF transformation are applied, and the trained model predicts one of the three stress levels. The interface can display the predicted label and a short explanation that the output is a machine learning classification result, not a medical diagnosis. This supports manual testing and makes the project easier to demonstrate during discussion.

IX. Ethical Considerations and Limitations

This project works with stress-related and mental-health-related text. Therefore, the output should not be treated as a clinical diagnosis or used as a replacement for professional support. The model only learns patterns from the training labels, and some labels are weak labels from query-based scraping. The system may also misunderstand sarcasm, context, cultural expression, or short ambiguous text.

Another limitation is that some source labels were mapped into stress classes using project-defined rules. For example, Anxious was mapped to High stress and Frustrated was mapped to Medium stress. These mappings are reasonable for a first course project, but they are simplifications. A future version should include manual annotation by multiple

reviewers, inter-annotator agreement, and more diverse stress-specific datasets.

X. Conclusion

This project developed a complete machine learning workflow for three-class stress-level text classification. The dataset was collected from a public text dataset and additional scraped text, then cleaned, merged, deduplicated, balanced, and transformed using TF-IDF. Five algorithms were implemented and evaluated on the same stratified split. Logistic Regression achieved the best overall performance, with 77.19% accuracy and 77.30% macro F1-score. The comparison showed that linear models are strong choices for TF-IDF-based text classification, while Naive Bayes and Decision Tree are useful baselines. Future work can improve the system by adding stronger manual labeling, more domain-specific stress data, hyperparameter tuning, cross-validation, and a more polished deployment interface.

Acknowledgment

The authors thank the CSCI417 Machine Intelligence course team for the project guidelines and rubric used to structure the dataset, implementation, evaluation, and final paper.

References

[1] CSCI417 Machine Intelligence, "Course Project Guidelines - Machine Learning Course," Fall 2025.

[2] CSCI417 Machine Intelligence, "CSCI417 Project Ideas," Medical Track, Stress Level Classifier.

[3] CSCI417 Machine Intelligence, "Phase 2 Project Rubric," 2026.

[4] MentalDistress: A multi-class social media text dataset for mental health-related emotion detection, Mendeley Data, DOI: 10.17632/b42wr437hg.2.

[5] Stack Exchange, "Stack Exchange API," API-based public question search workflow, accessed for project data collection.

[6] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[7] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.